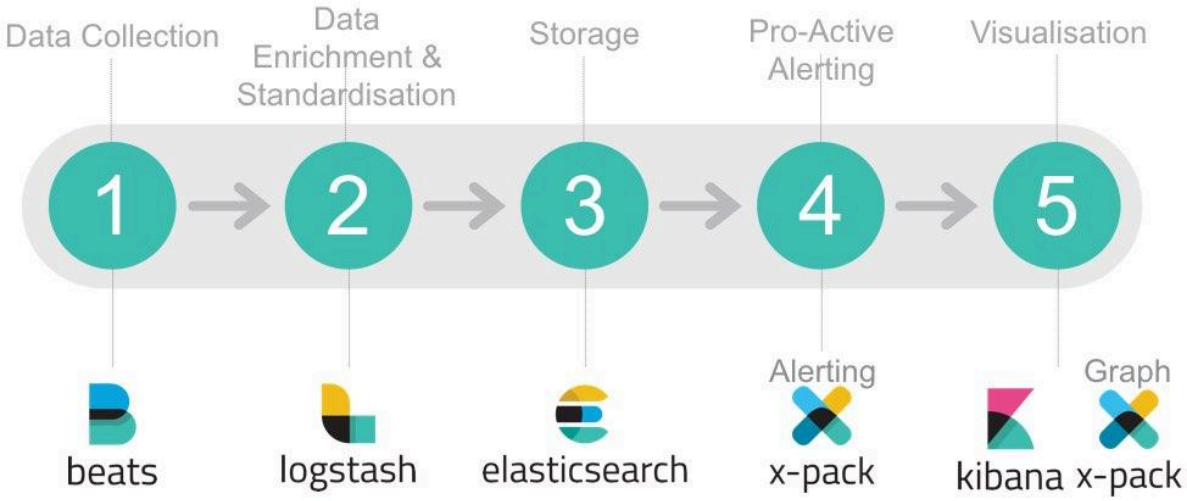
Getting Started: Elastic Workflows - Elastic Stack

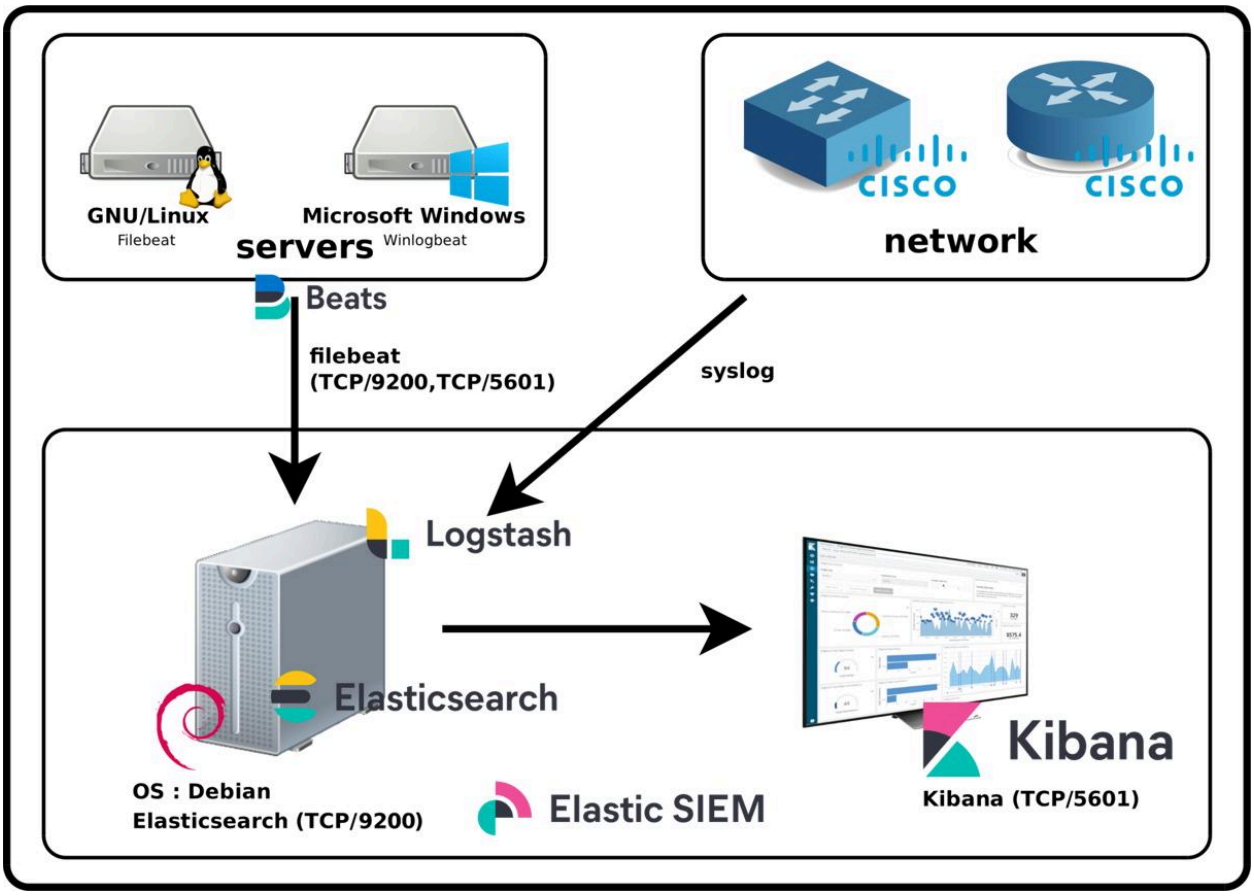
the **Elastic Stack workflow** means the process of taking security data from your systems, sending it to Elastic, detecting suspicious activity, investigating it, and responding to incidents.

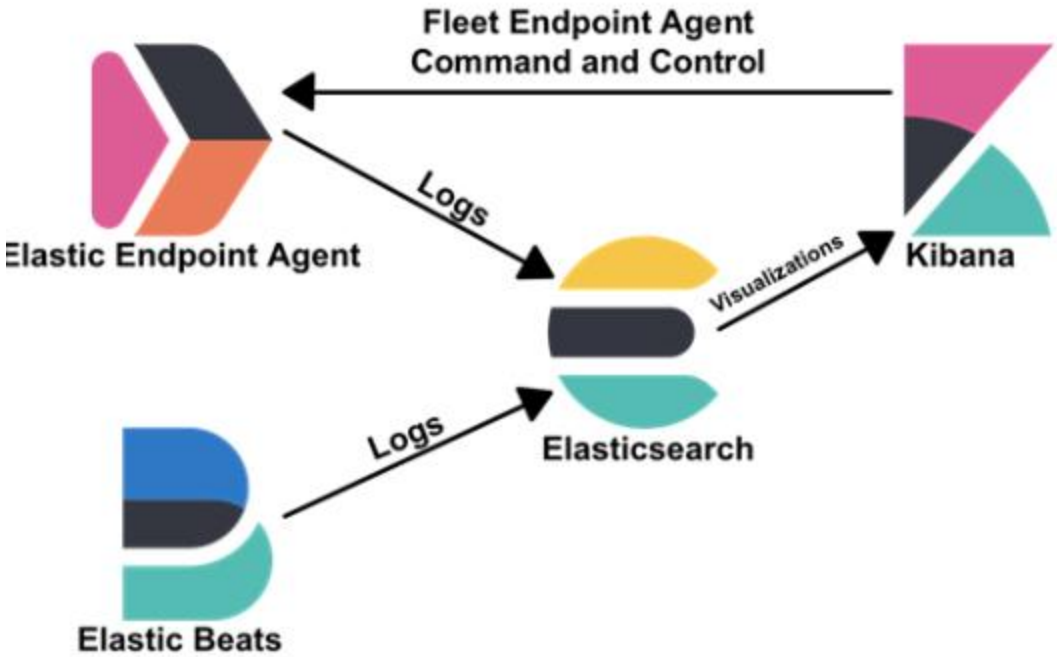
Think of it like this:

Logs → Collect → Store → Search → Detect → Investigate → Respond

Security and Threat Detection with the Elastic Stack




GNU/Linux
Filebeat
Microsoft Windows
Winlogbeat
servers
network
Beats
filebeat
(TCP/9200,TCP/5601)
syslog
Logstash
Elasticsearch
Kibana
OS : Debian
Elasticsearch (TCP/9200)
Elastic SIEM
Kibana (TCP/5601)


Fleet Endpoint Agent
Command and Control
Logs
Elastic Endpoint Agent
Visualizations
Kibana
Elasticsearch
Logs
Elastic Beats

1. Collect logs

Your systems generate logs:

- Windows Event Logs
- Linux logs
- Firewall logs
- DNS logs
- Web server logs
- VPN logs
- Endpoint/EDR logs
- Network traffic

For example, a Windows computer records:

```
User: Ahmed
Event: Failed login
Source IP: 192.168.1.50
Time: 10:32 PM
```

---

2. Send logs to Elastic

Tools such as **Elastic Agent** collect the logs and send them to Elasticsearch.

A simplified flow is:

```
Windows
Linux
Firewall
Servers
   |
   ▼
Elastic Agent
   |
```

```
   ▼
Elasticsearch
```

3. Elasticsearch stores and indexes the data

Elasticsearch is the main search and storage engine.

It indexes the logs so a SOC analyst can quickly search things like:

```
failed login
source.ip: 192.168.1.50
event.category: authentication
```

4. Kibana lets you see the data

Kibana is the interface where you search and visualize your logs.

For a SOC analyst, you might have dashboards showing:

```
Failed Logins       1,245
Successful Logins   8,430
Malware Alerts         12
Critical Alerts         3
```

You can also investigate individual events.

5. Detection rules look for attacks

Elastic Security can run **detection rules** against your logs.

For example:

```
5 failed logins
        ↓
same username
        ↓
within 1 minute
        ↓
Detection Rule
```

```
        ↓
ALERT
```

Another example:

```
PowerShell execution
        +
Encoded command
        +
Suspicious parent process
        ↓
Possible attack
        ↓
SOC Alert
```

---

6. SOC analyst investigates the alert

This is where **you as a SOC L1 analyst** would spend a lot of time.

You receive:

> **Alert: Multiple Failed Login Attempts**

You investigate:

```
Who?       admin
Source IP? 185.x.x.x
When?      02:31 AM
How many?  50 attempts
Target?    Windows Server
```

Then you check related logs:

```
Authentication logs
       ↓
Endpoint logs
       ↓
Network logs
       ↓
DNS logs
```

```
        ↓
Threat intelligence
```

You try to determine:

Is this a real attack or a false positive?

---

7. Escalate or respond

If it is a real incident:

```
Alert
  ↓
Investigate
  ↓
Confirm attack
  ↓
Contain / escalate
  ↓
Document incident
```

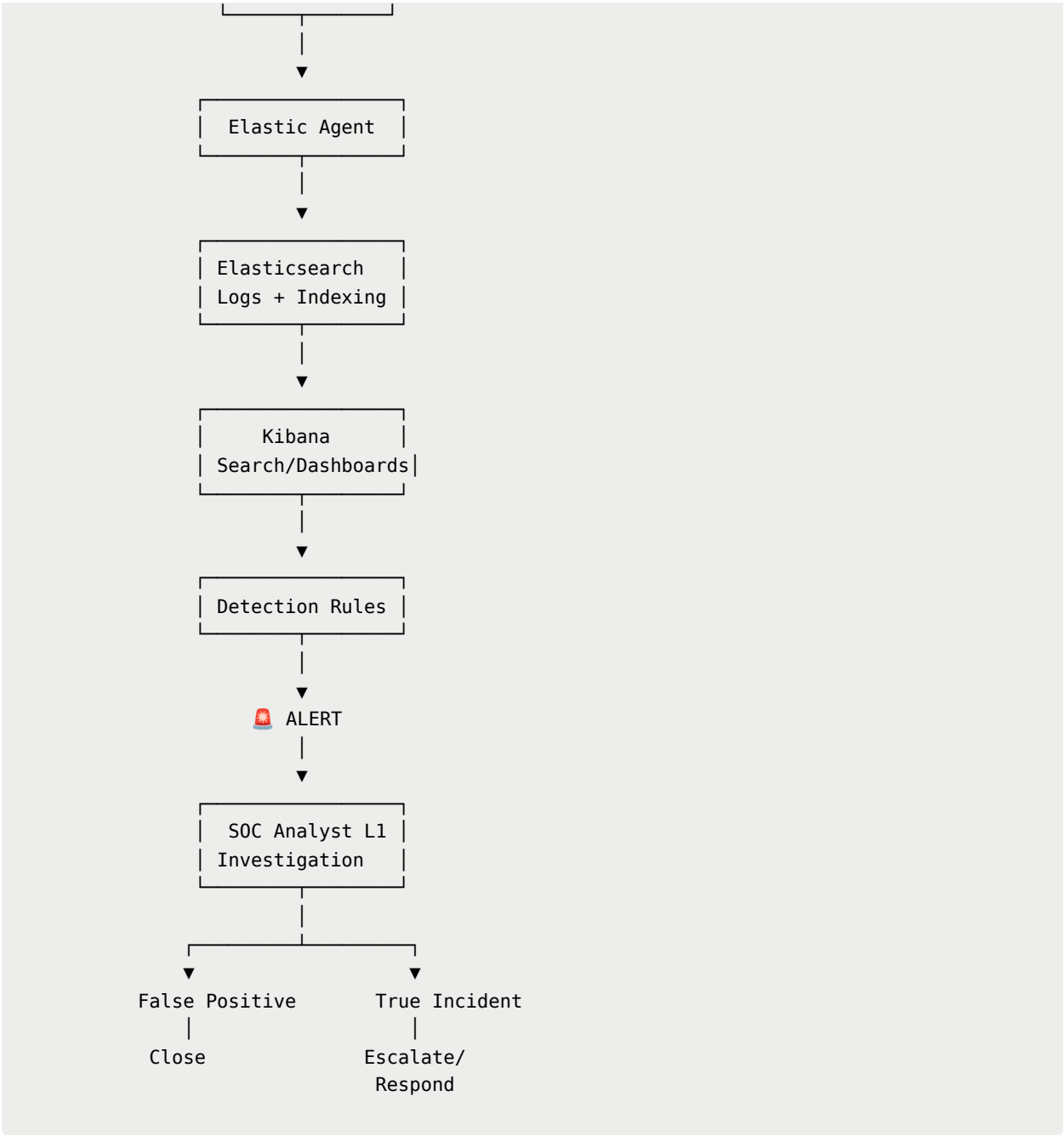
For an L1 analyst, you might:

- Block an IP according to your organization's procedure
- Disable/lock a compromised account
- Isolate an endpoint if authorized
- Collect additional evidence
- Escalate to L2/Incident Response
- Document what happened

The complete Elastic SOC workflow

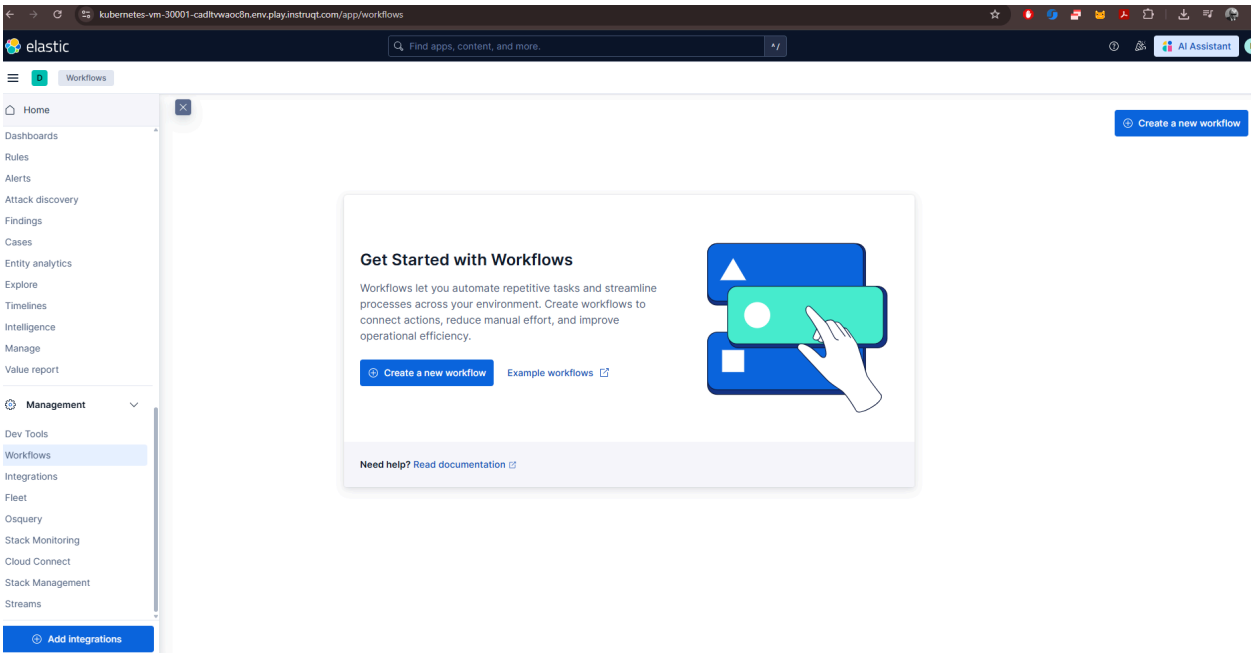
Remember this diagram:

```
                ┌──────────────┐
                │   Endpoints  │
                │   Servers    │
                │   Firewalls  │
                │   DNS/VPN    │
```

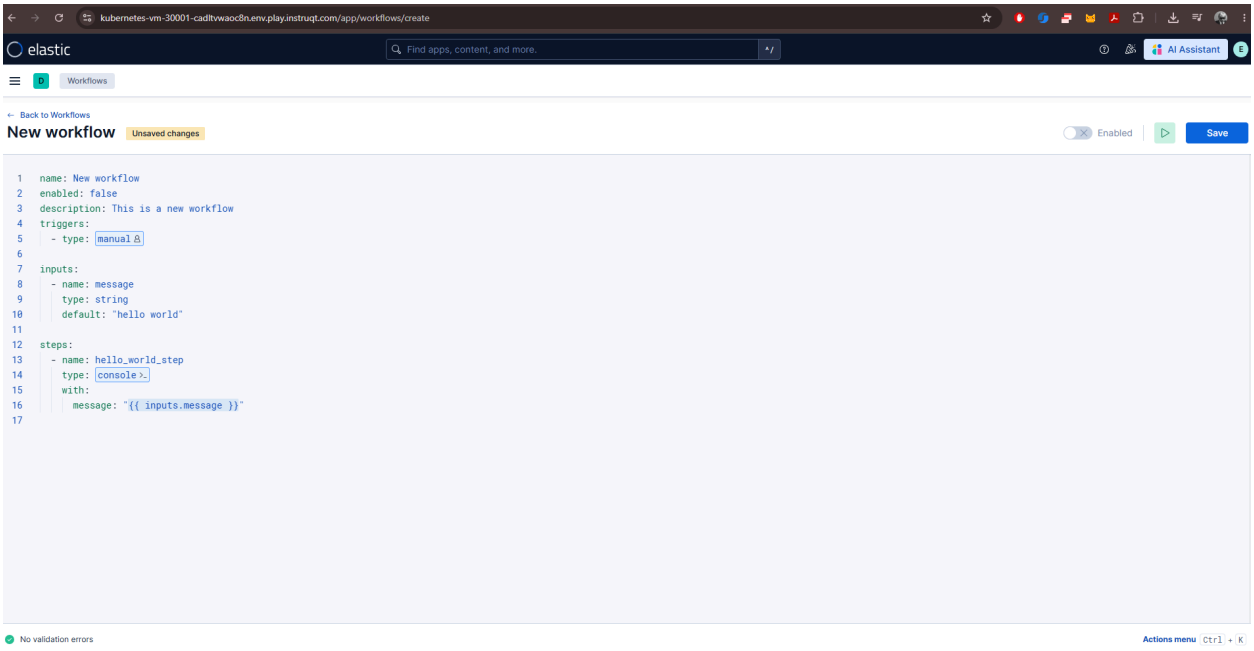


Building your first Workflow:

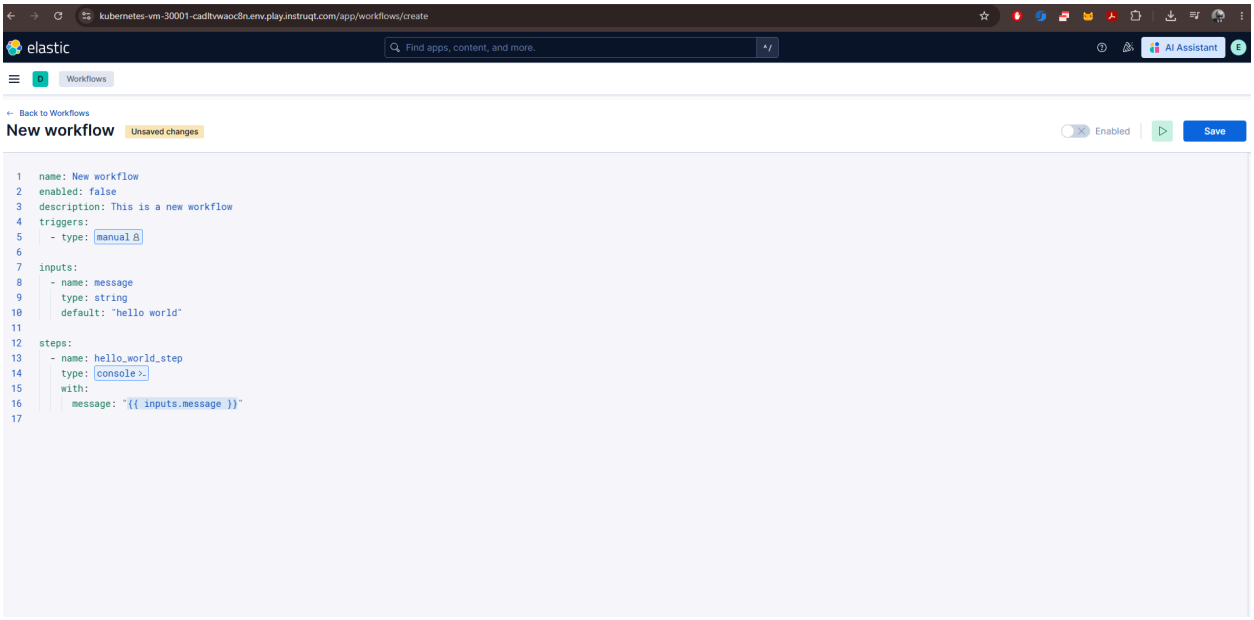
1. From the left navigation menu, navigate to **Management** → **Workflows**.



1. Click **Create a new workflow**.



Kibana automatically loads a default template in the editor pane.



1. Take a moment to read through the workflow template to understand how each section is structured.

1. Workflow name

name: Suspicious Login Detection

1. Workflow name

```
name: Suspicious Login Detection
```

Enabled

```
enabled: false
```

This means:

> ❌ The workflow is currently **OFF**.

If you change it to:

```
enabled: true
```

then the workflow can actually run.

3. Description

```
description: Detect suspicious Windows login activity
```

It doesn't perform an action. It's just documentation.

4. Triggers

```
triggers:
  - type: manual
```

This is **very important**.

A trigger answers:

> **"When should my workflow start?"**

Your workflow says:

```
type: manual
```

So it means:

> 👤 **A person must manually start the workflow.**

For example:

```
Security alert
      ↓
Workflow starts
      ↓
Investigate / notify / respond
```

5. Inputs

```
inputs:
  - name: message
    type: string
    default:"Suspicious Login Detection"
```

Inputs are basically **information that you give to the workflow**.

Think of a function in programming:

```
function sayHello(message)
```

The `message` is the input.

Here:

```
name: message
```

means the input is called **message**.

```
type: string
```

This means the input contains **text**

```
Suspicious login detected
```

`default`

```
default:"hello world"
```

This means:

> If you don't provide a message, use **"hello world"**.

So:

```
message = "Suspicious login detected"
```

6. Steps

Now we reach the actual **work**.

```
steps:
```

A workflow can have one or many steps.

Think:

```
Workflow
   ↓
Step 1
   ↓
Step 2
   ↓
Step 3
```

Your workflow currently has only **one step**.

---

7. Step name

```
- name: hello_world_step
```

This is simply the name of the step.

You could call it:

```
- name: print_message
```

or:

```
- name: investigate_alert
```

The name helps you understand what the step does.

8. Type: console

```
type: console
```

This tells Elastic:

> "For this step, display/output something in the console."

In your example, the workflow is basically saying:

> **Print the message.**

---

9. `with`

```
with:
```

`with` contains the **settings/data that we give to this step**.

Your step receives:

```
message:"{{ inputs.message }}"
```

This is the most important part to understand.

---

10. What does `{{ inputs.message }}` mean?

This:

```
{{ inputs.message }}
```

means:

> **Take the value of the `message` input.**

Remember we had:

```
inputs:
  - name: message
    type: string
    default:"Suspicious login detected"
```

So:

```
inputs.message
```

means:

```
the message input
```

And:

```
{{ inputs.message }}
```

means:

```
put the value of message here
```

A real SOC workflow could look more like:

```
Windows security alert
        ↓
Workflow triggered
        ↓
Get username
        ↓
Get source IP
        ↓
Check IP reputation
        ↓
Create investigation
        ↓
Notify SOC analyst
```

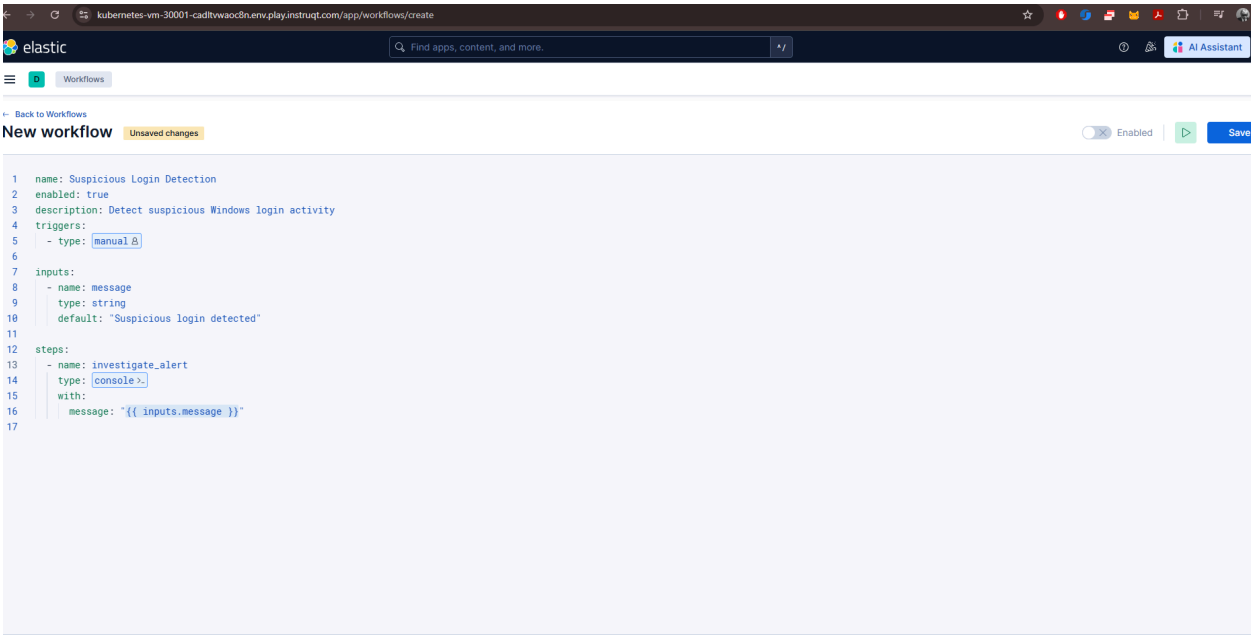
```
🚨 Failed login detected
        ↓
IP = 185.xxx.xxx.xxx
        ↓
Check threat intelligence
        ↓
Is IP malicious?
       / \
```

```
    YES  NO
     ↓    ↓
  Alert  Close
  SOC
```

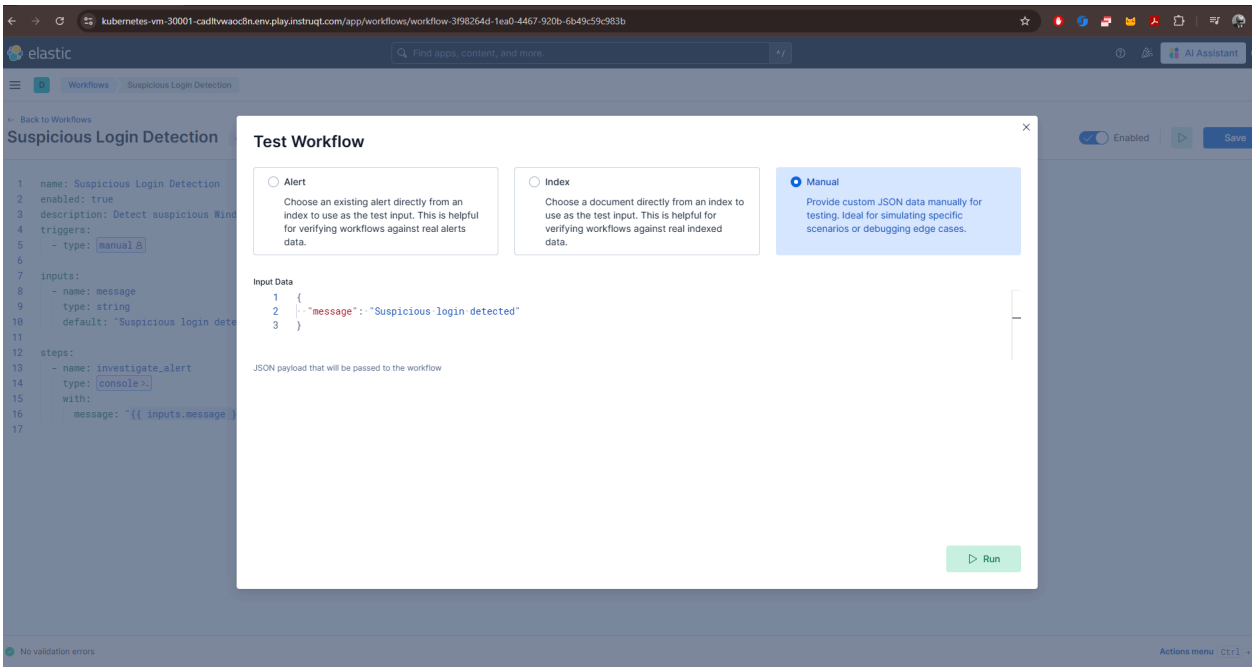
Easy meaning of every section

Code	Easy meaning
`name`	Name of workflow
`enabled`	ON/OFF
`description`	What the workflow is for
`triggers`	When workflow starts
`manual`	Start it yourself
`inputs`	Information given to workflow
`name: message`	Input is called message
`type: string`	Input is text
`default`	Value used if nothing is provided
`steps`	Things the workflow does
`name`	Name of a step
`type: console`	Output/display something
`with`	Data/settings given to the step
`{{ inputs.message }}`	Use the message provided as input

customize workflow



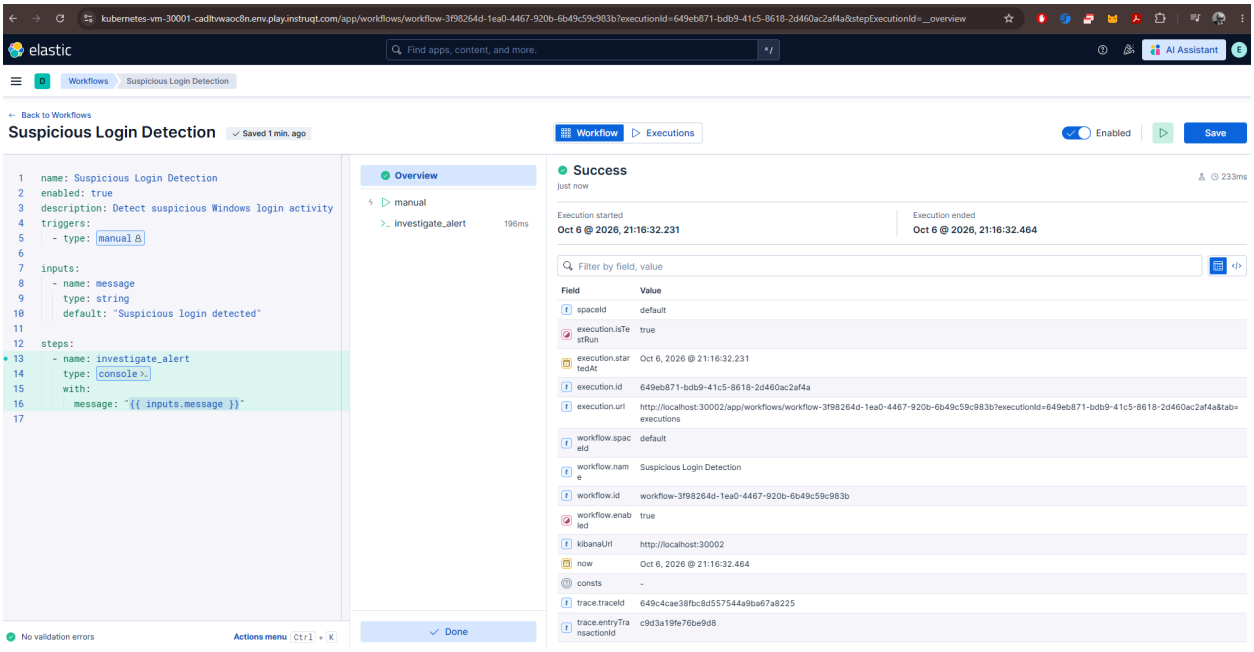
Click the green run icon on the top right to execute the workflow



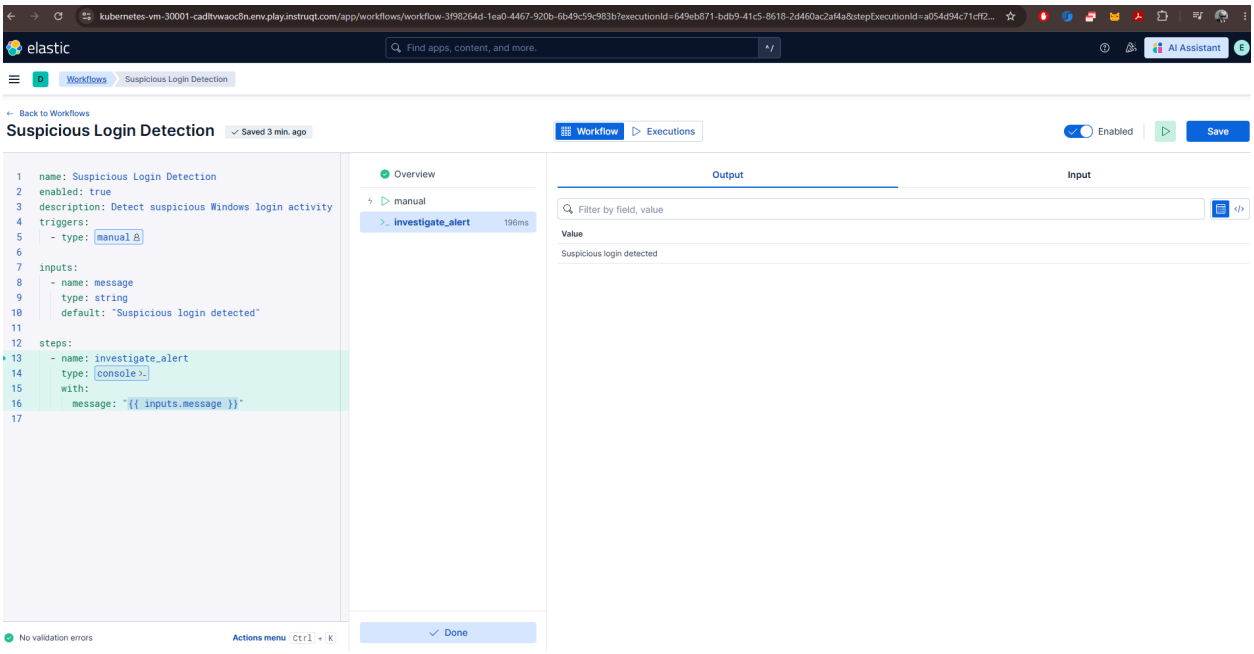
1. As your workflow runs, execution logs display in a panel next to your workflow. In the panel, you can find:

- **Real-time execution logs:** Each step appears as it executes.

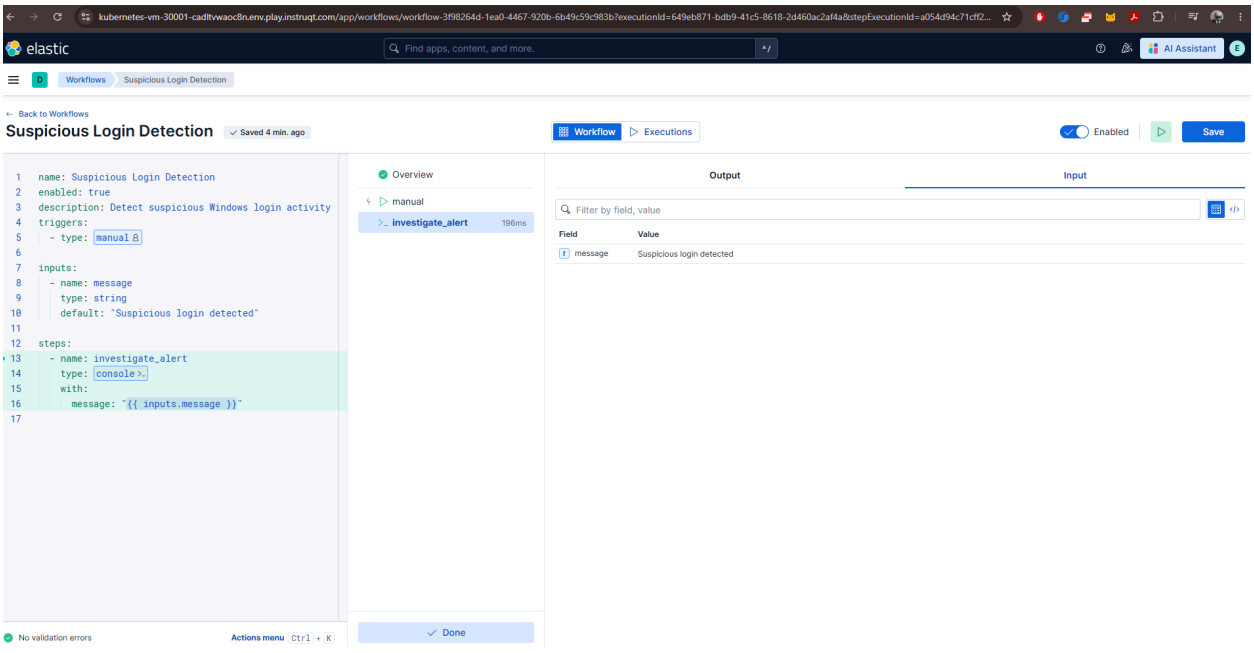
- **Workflow status indicators:** Green for success, red for failures, and timestamps for duration.
- **Expandable step details:** Click any step to see its input, output, and timeline.



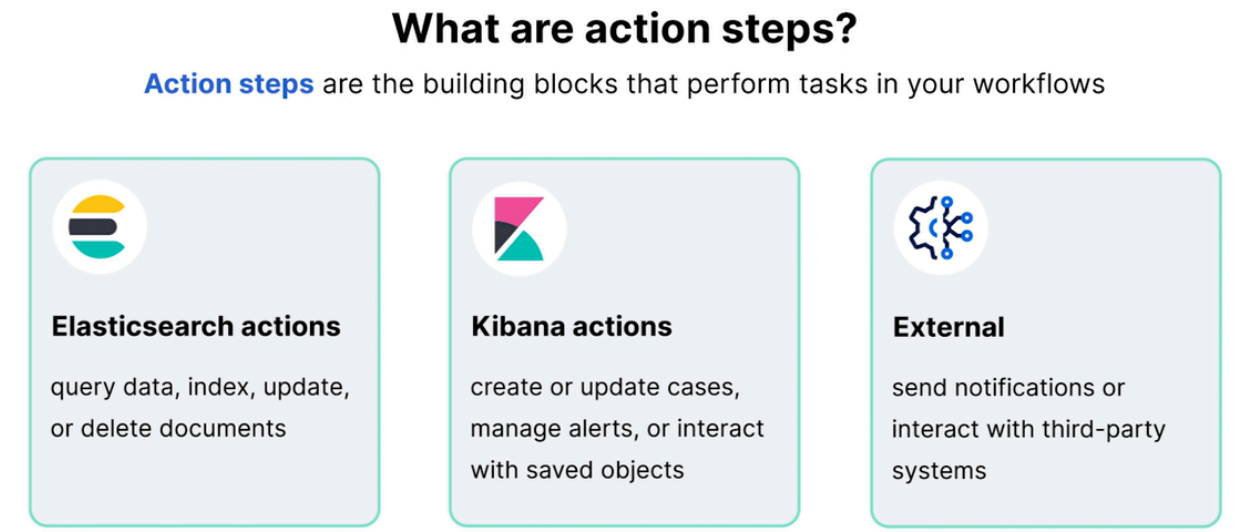
1. Click on the investigate_alert in the panel and confirm the output shows: Suspicious login detected



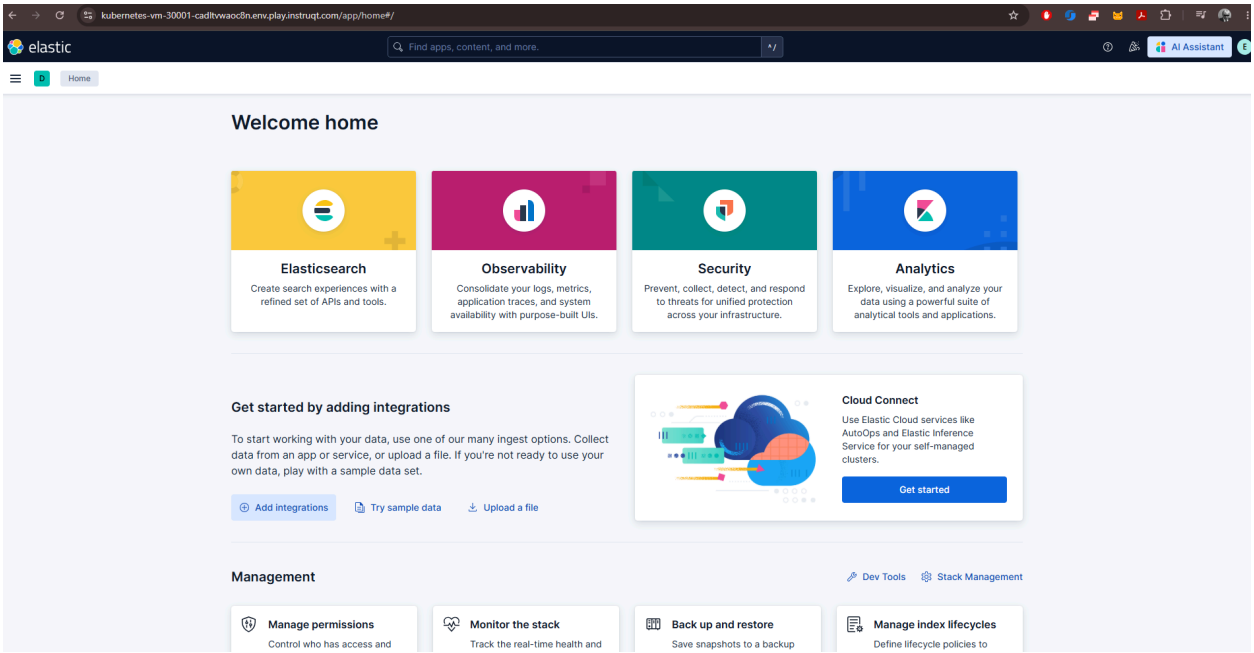
The input named **message** defines this value. Then in the console step, the workflow references that input using double curly braces. When the workflow runs, it replaces that reference with the actual input value, which is what gets displayed in the output.



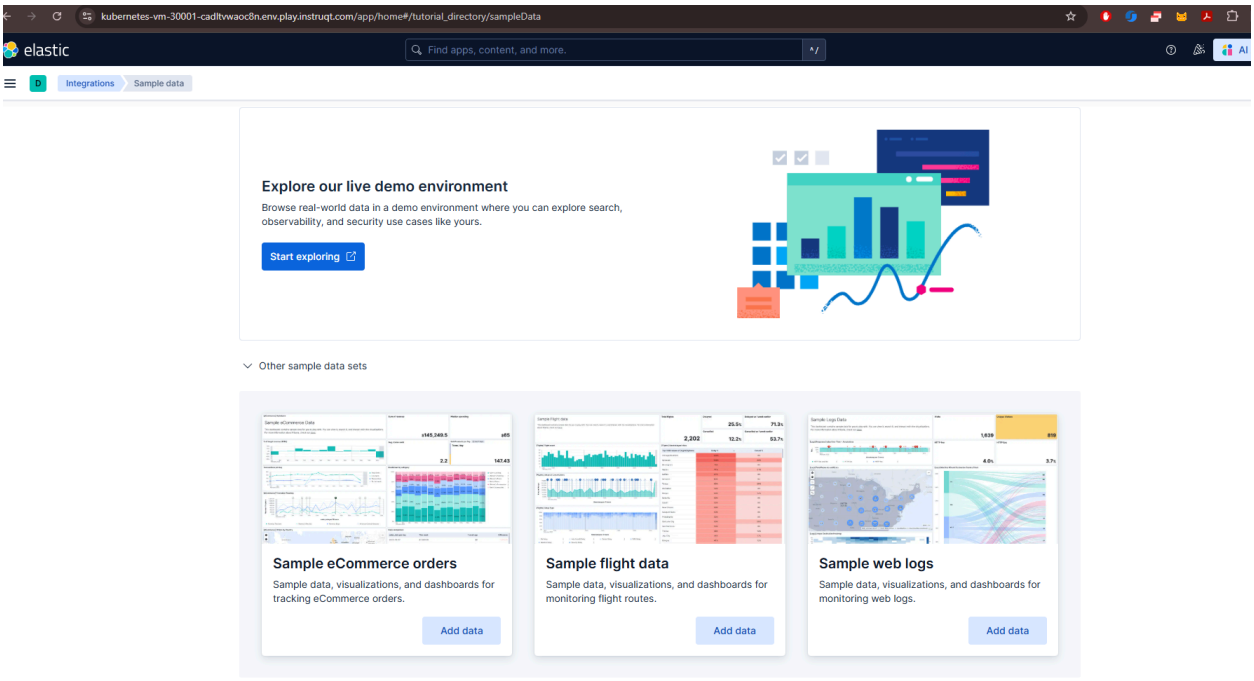
Querying Data with action steps:



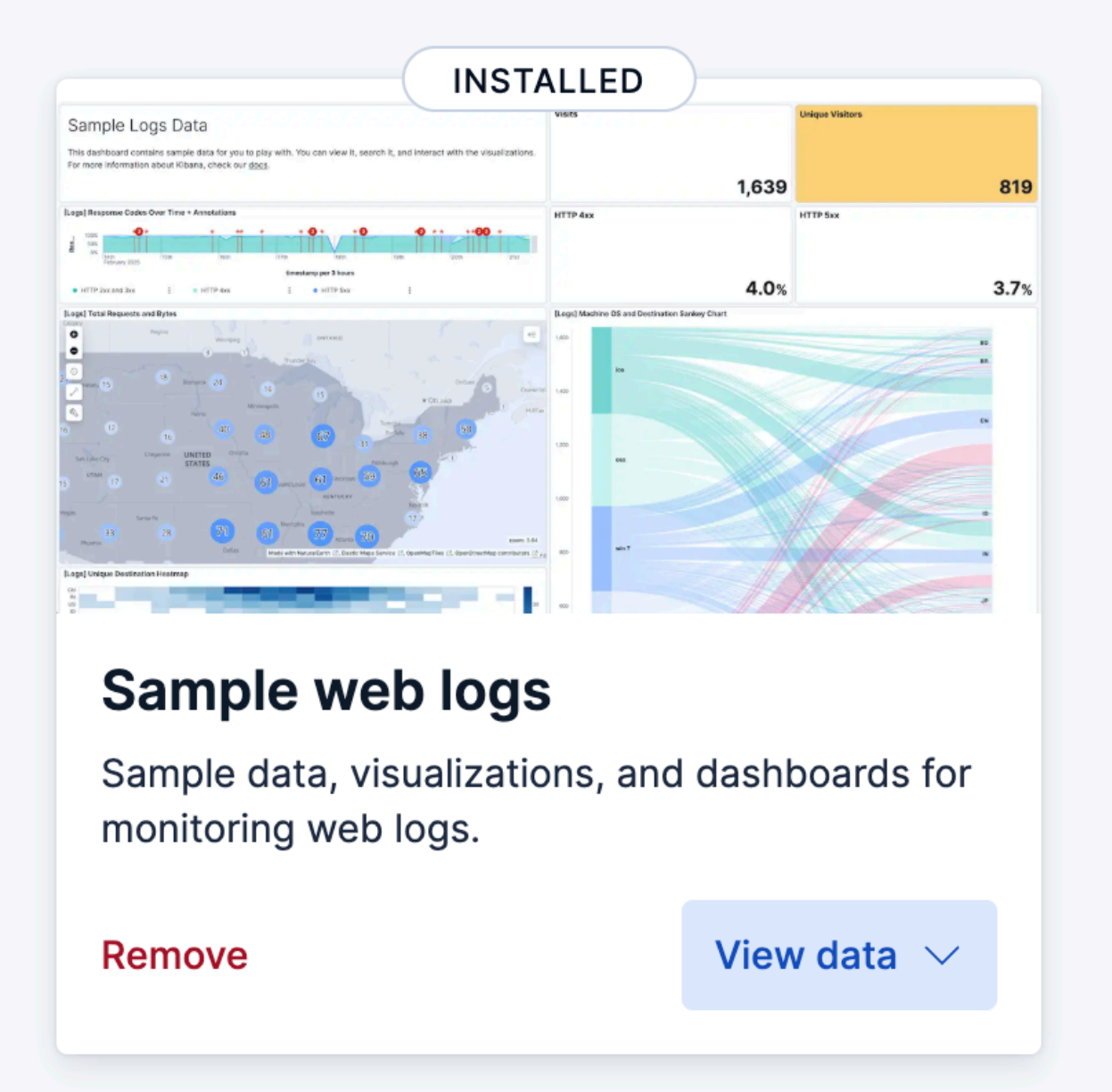
1. On the home page, click on **Try sample data**.



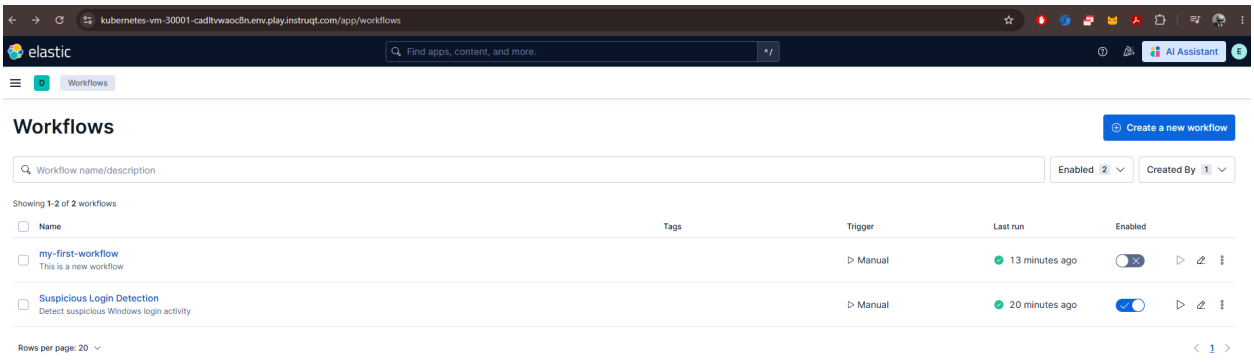
1. Click on **Other sample data sets**.



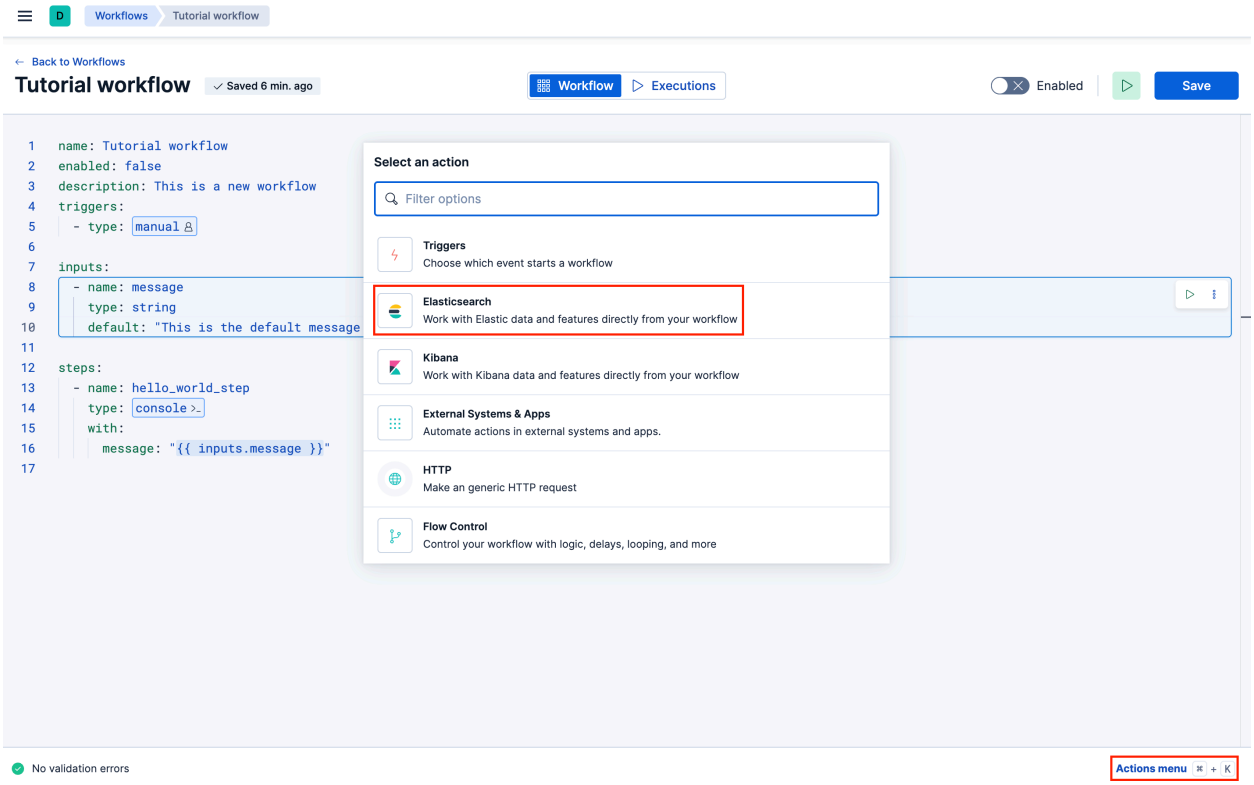
1. Click on **Add data** under **Sample web logs**.



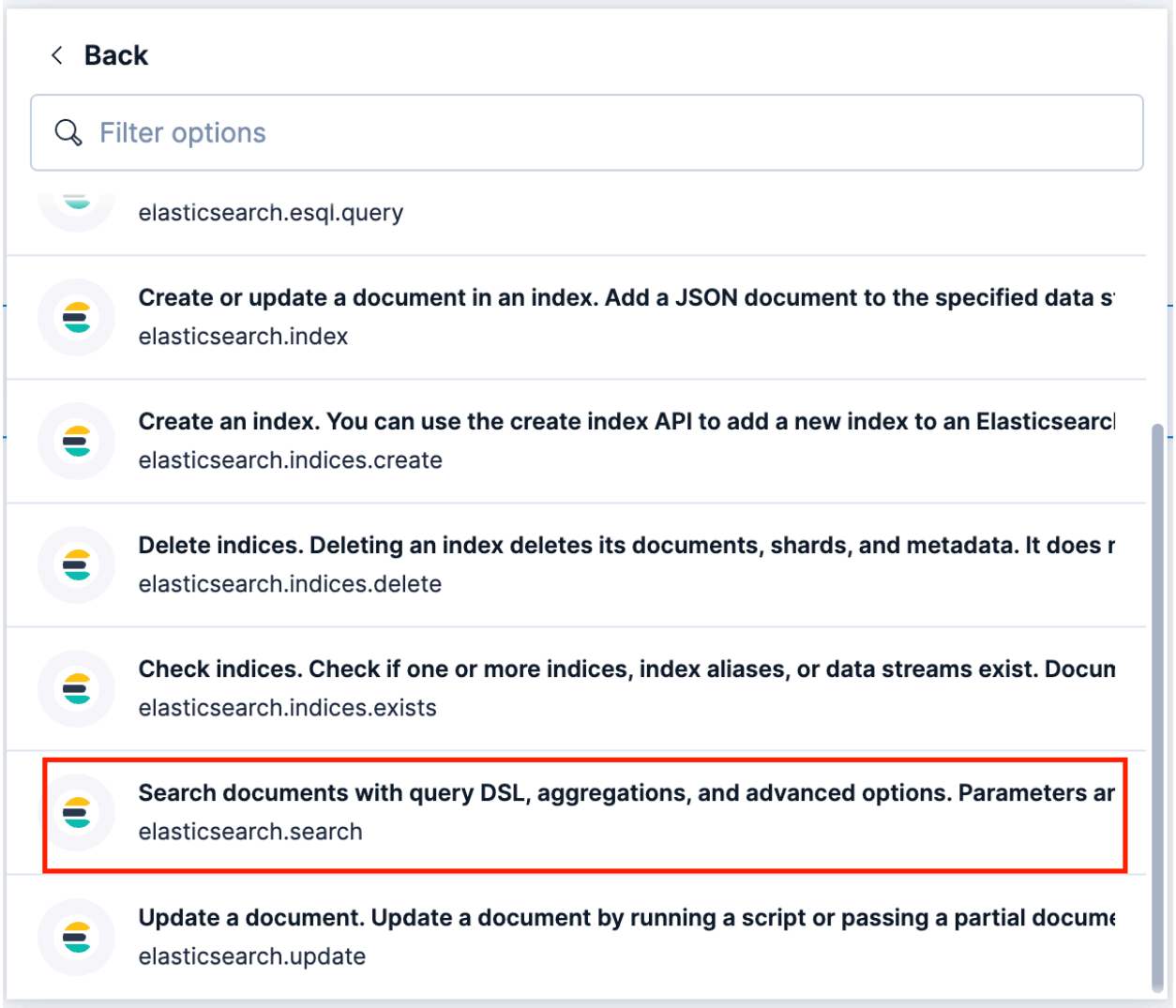
1. Navigate to **Management** → **Workflows** and click **Tutorial workflow**.



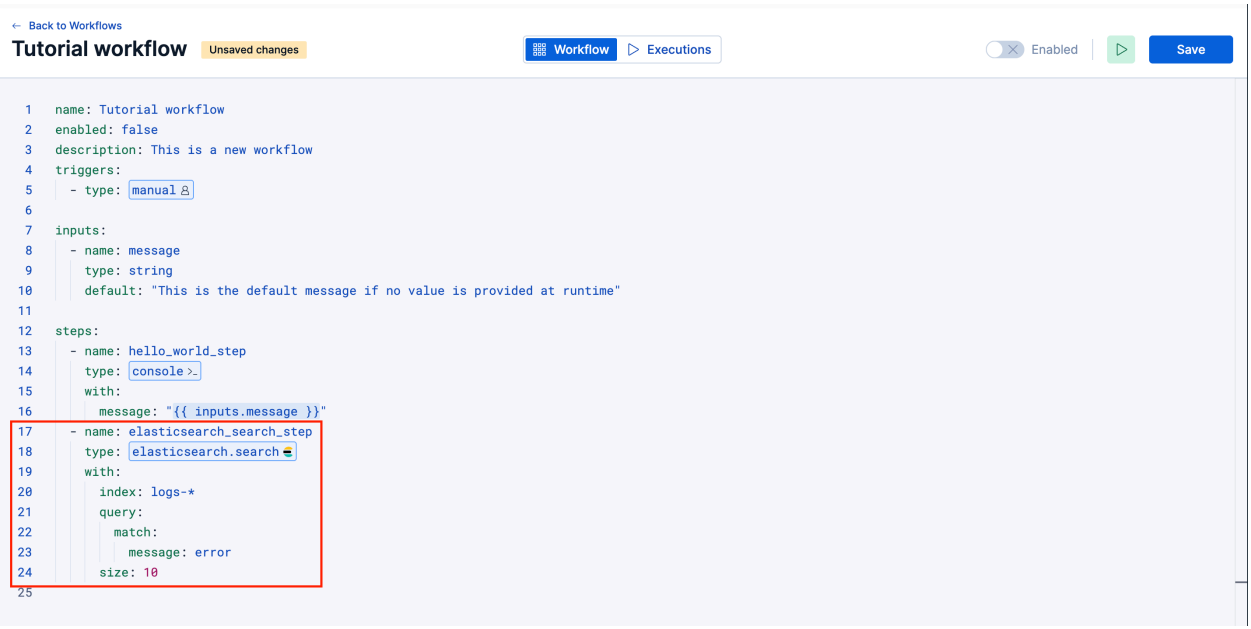
While in the editor pane, open the **Actions menu**



Scroll down and click the option that begins with **"Search documents...**



1. Notice how the step structure was added to the workflow.

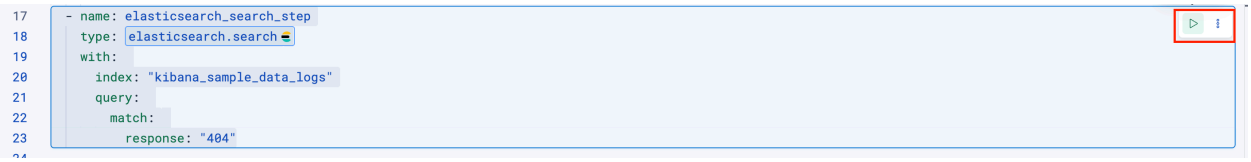


1. Edit the **elasticsearch_search_step** to search for failed web requests in the **kibana_sample_data_logs** index. The step should look like the following:

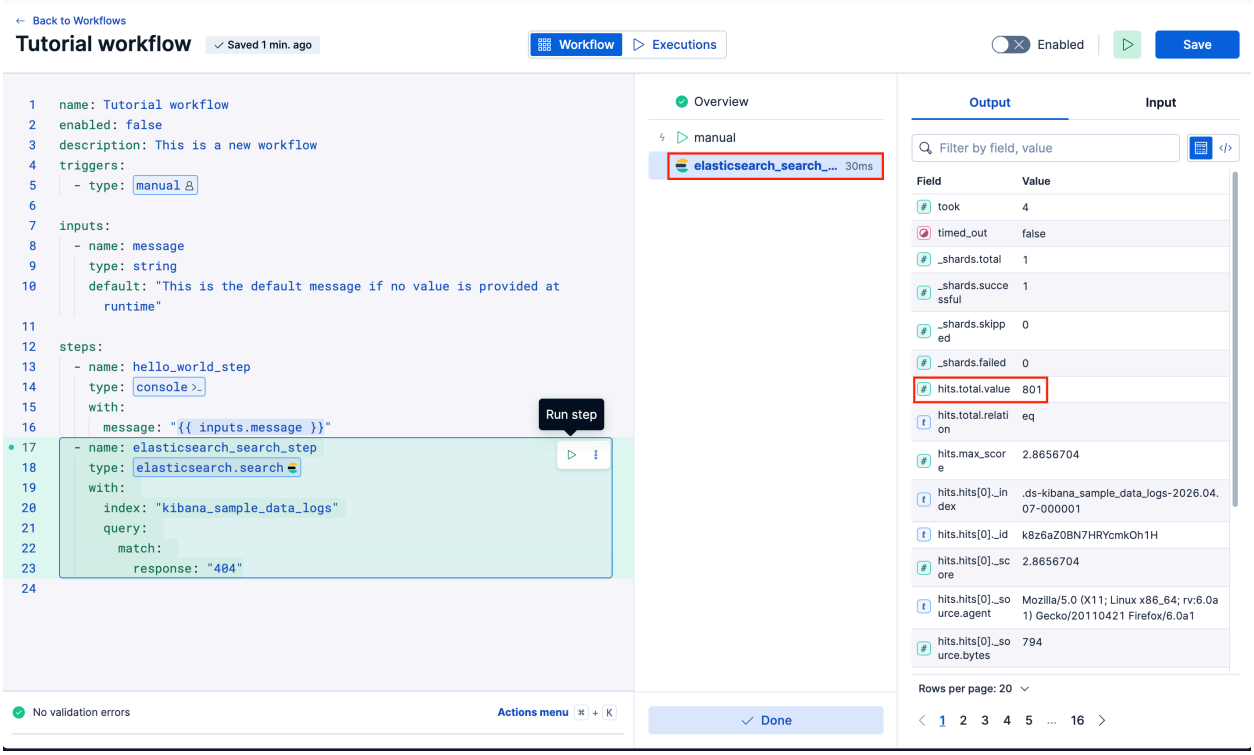
- name: elasticsearch_search_step
  type: elasticsearch.search
  with:
  index: "kibana_sample_data_logs"
  query:
  match:
  response: "404"

This query filters for documents where the response code is **404**, which typically indicates a failed request.

1. Run the **elasticsearch_search_step** and view the results in the execution panel.



1. Click on the **elasticsearch_search_step** and explore the output.



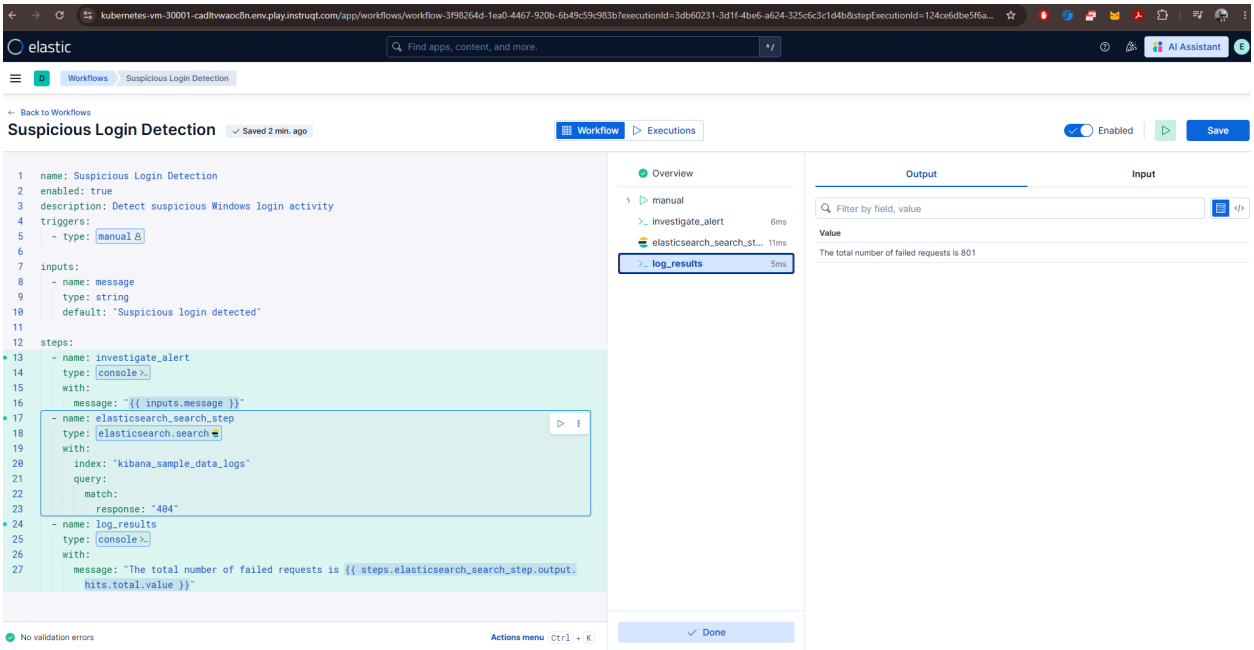
1. Locate `hits.total.value` in the results. This is the search response that contains the total number of matching documents from the **kibana_sample_data_logs** index.
2. Add a new step to display a value from the search results.

- name: log_results
  type: console
  with:
  message: "The total number of failed requests is {{ steps.elasticsearch_search_step.output.hits.total.value }}"

```yaml
- name: elasticsearch_search_step
  type: elasticsearch.search
  with:
    index: "kibana_sample_data_logs"
    query:
      match:
        response: "404"
- name: log_results
  type: console
  with:
    message: "The total number of failed requests is {{ steps.elasticsearch_search_step.output.
      hits.total.value }}"
```

1. Click **Save**, then click the green **Run** icon.
2. Leave the default message and click **Run**.
3. Watch the execution panel as all three steps run in sequence.

Click **log_results** and confirm the output displays the number of failed requests from the search results.

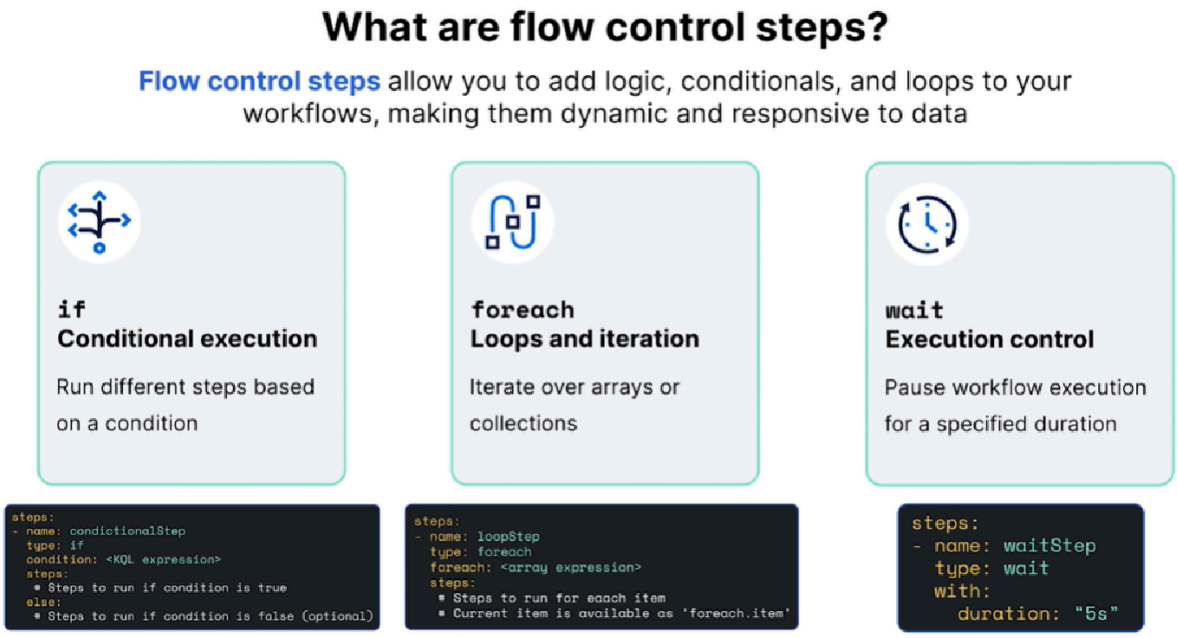


In this lesson, i learned:

- Action steps perform tasks in a workflow, such as querying data and interacting with systems

- A search step can be added to retrieve data from an Elasticsearch index
- Workflow steps produce structured results that can be explored and understood
- Results can be referenced using template syntax to display output and pass data between steps

Building Dynamic Workflows with Flow Control Steps:

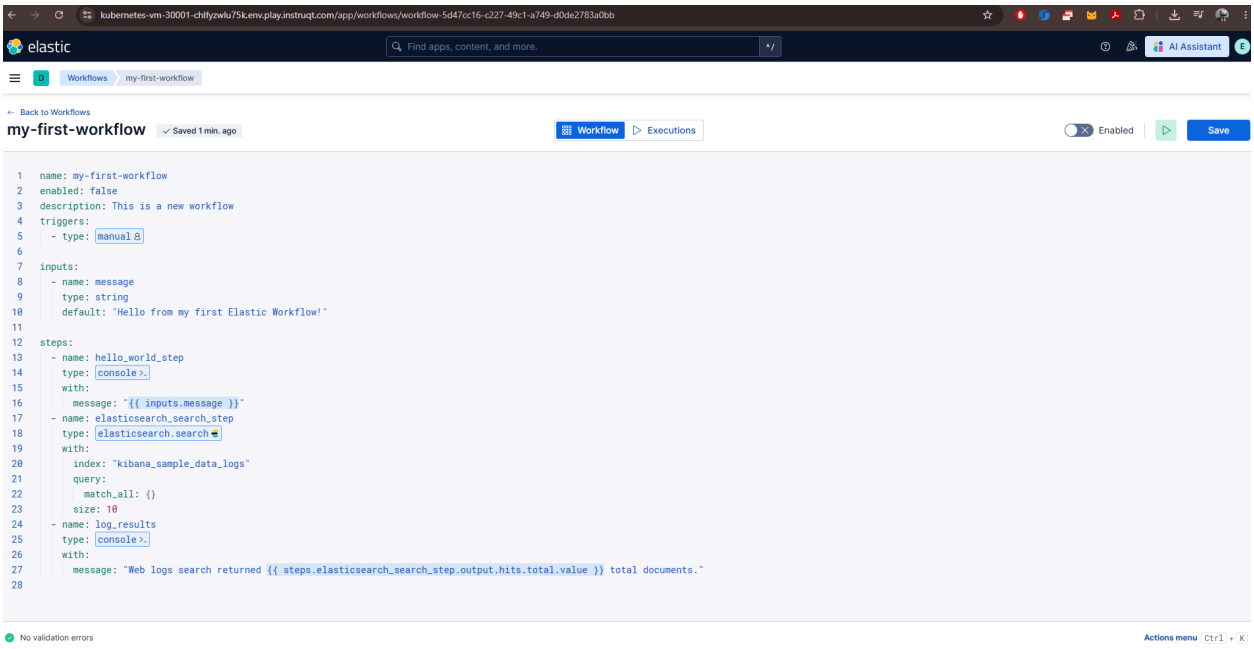


1. Navigate to **Management** → **Workflows** and click **Tutorial workflow**.

This workflow should already contain:

- A manual trigger
- A **hello_world_step**
- An **elasticsearch_search_step** that searches the **kibana_sample_data_logs** index for response: "404"
- A **log_results** step that displays the total number of failed requests

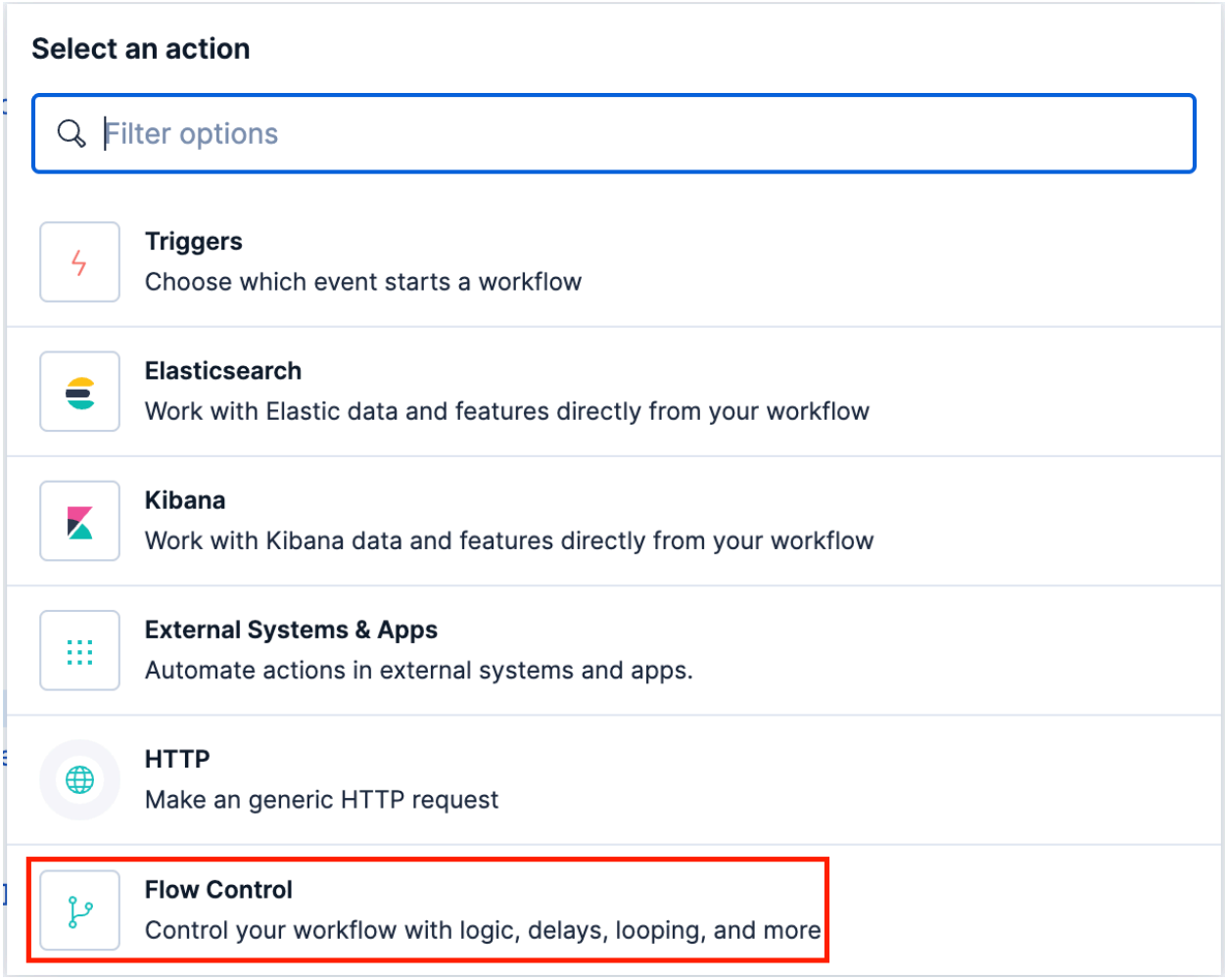
1. Before adding flow control, update the search step so the workflow only returns a small set of results. Add **size: 5** to the **elasticsearch_search_step**:

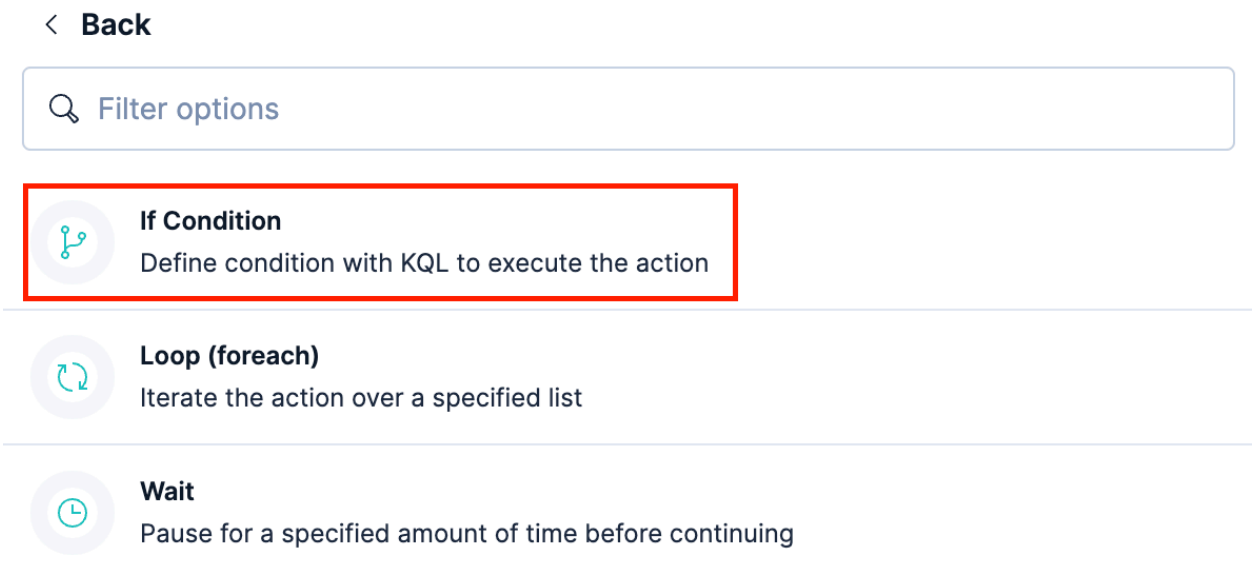



```
  - name: elasticsearch_search_step
    type: elasticsearch.search
    with:
      index: "kibana_sample_data_logs"
      query:
        match:
          response: "404"
      size: 5
```

This makes the loop easier to read in the execution panel because it will process only five failed requests.

1. Remove the existing **log_results** step. We're going to replace it with an if step that can handle both true and false outcomes.


Select an action
Filter options
Triggers
Choose which event starts a workflow
Elasticsearch
Work with Elastic data and features directly from your workflow
Kibana
Work with Kibana data and features directly from your workflow
External Systems & Apps
Automate actions in external systems and apps.
HTTP
Make an generic HTTP request
Flow Control
Control your workflow with logic, delays, looping, and more


Back
Filter options
If Condition
Define condition with KQL to execute the action
Loop (foreach)
Iterate the action over a specified list
Wait
Pause for a specified amount of time before continuing

1. Edit the condition so it checks whether the search returned any failed requests:

```
 - name: if_step
    type: if
    condition: steps.elasticsearch_search_step.output.hits.to
tal.value > 0
```

This condition reads the output of the search step. **hits.total.value** is the number of documents that matched the query. If it's greater than 0, the **true branch** runs.

1. Inside the **if_step** add a console step to display how many failed requests were found with a message. This will be part of the **true branch**.

```
- - name: if_step
    type: if
    condition: steps.elasticsearch_search_step.output.hits.to
tal.value > 0
    steps:
      - name: failed_requests_found
        type: console
        with:
          message: "Found {{ steps.elasticsearch_search_step.
output.hits.total.value }} failed requests. Processing the fi
rst 5 results..."
```

1. Add a wait step inside the true branch after **failed_requests_found**. This introduces a short pause before the workflow starts processing the results one by one.

```
- name: pause_before_processing
  type: wait
  with:
    duration: "3s"
```

1. Add a **foreach** step after **pause_before_processing**. Set it to iterate over the search results using:

```
   - name: process_each_log
       type: foreach
       foreach: "{{ steps.elasticsearch_search_step.output.h
its.hits | json }}"
```

hits.hits is the array of returned documents. The **| json** filter converts it into a format the **foreach** step can iterate over.

1. Inside the **foreach** step, add a console step that logs the request path.

```
      - name: process_each_log
        type: foreach
        foreach: "{{ steps.elasticsearch_search_step.output.h
its.hits | json }}"
        steps:
          - name: log_url
            type: console
            with:
              message: "Failed request path: {{ foreach.item.
_source.url }}"
```

foreach.item refers to the current document in the loop. `_source.url` is the URL field from that document's source.

1. Add an else branch with a console step that indicates no results were found with a message. This allows the workflow to respond differently based on the data and will run the false branch.

```
    else:
      - name: no_failed_requests_found
        type: console
        with:
```

```
          message: "No failed requests were found in the samp
le web logs."
```

full YAML should now look like this:

```yaml
name: Tutorial workflow
enabled: false
description: This is a new workflow
triggers:
  - type: manual

inputs:
  - name: message
    type: string
    default: "This is the default message if no value is prov
ided at runtime"

steps:
  - name: hello_world_step
    type: console
    with:
      message: "{{ inputs.message }}"

  - name: elasticsearch_search_step
    type: elasticsearch.search
    with:
      index: "kibana_sample_data_logs"
      query:
        match:
          response: "404"
      size: 5
  - name: if_step
    type: if
    condition: steps.elasticsearch_search_step.output.hits.to
tal.value > 0
    steps:
```

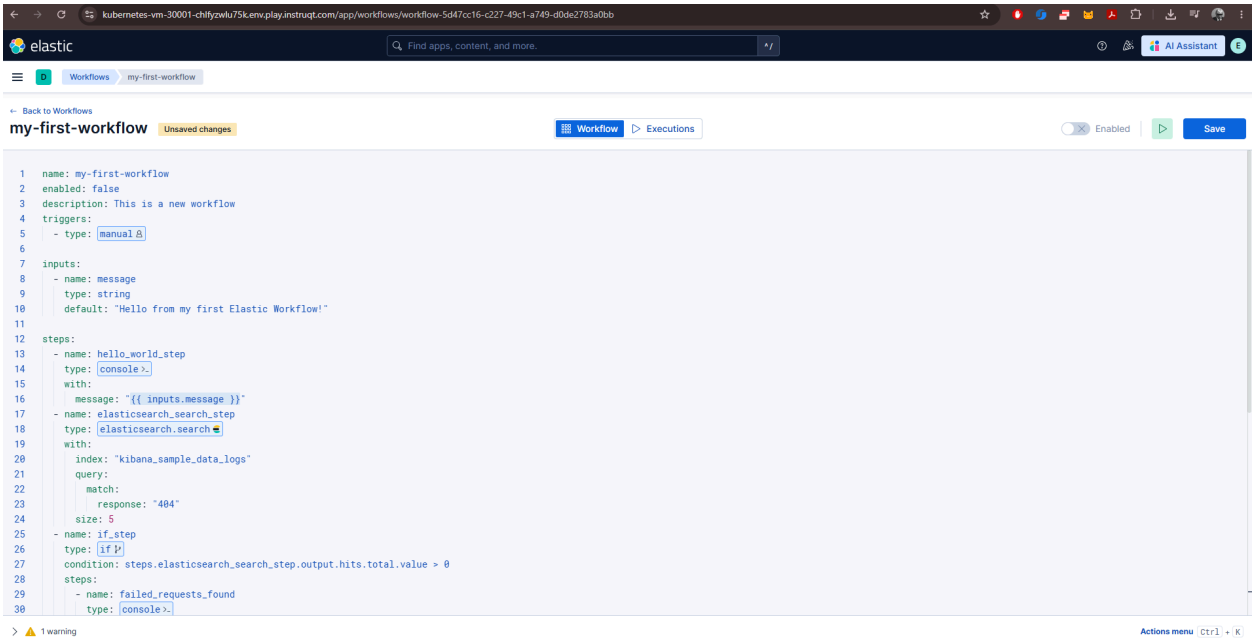
```
      - name: failed_requests_found
        type: console
        with:
          message: "Found {{ steps.elasticsearch_search_step.
output.hits.total.value }} failed requests. Processing the fi
rst 5 results..."
      - name: pause_before_processing
        type: wait
        with:
          duration: "3s"
      - name: process_each_log
        type: foreach
        foreach: "{{ steps.elasticsearch_search_step.output.h
its.hits | json }}"
        steps:
          - name: log_url
            type: console
            with:
              message: "Failed request path: {{ foreach.item.
_source.url }}"
    else:
      - name: no_failed_requests_found
        type: console
        with:
          message: "No failed requests were found in the samp
le web logs."
```

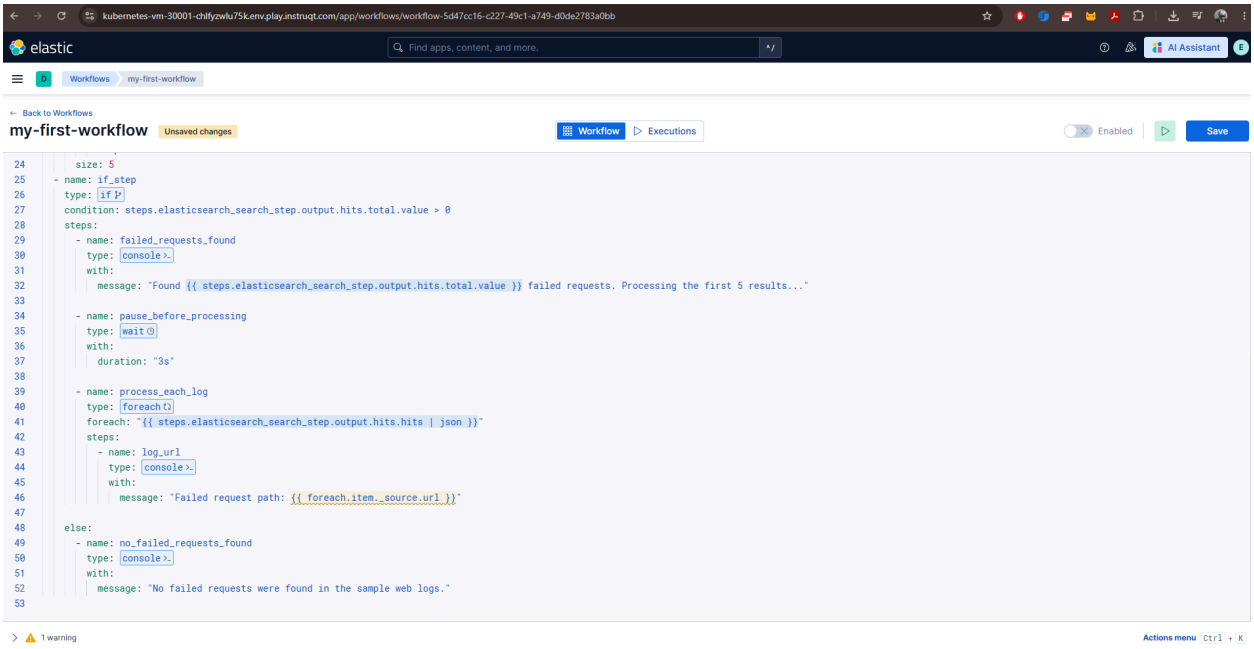
1. Click **Save**, then click the green **Run** icon. Leave the default message and click **Run**.
2. Watch the execution panel as the workflow runs.

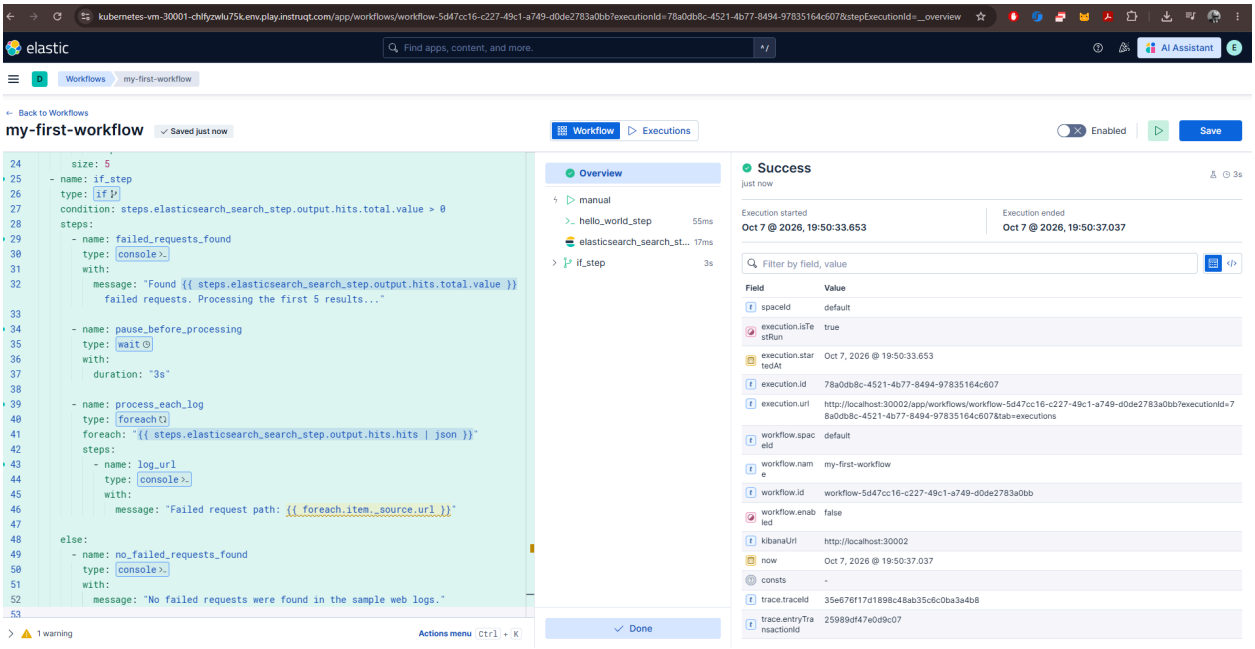
When complete:

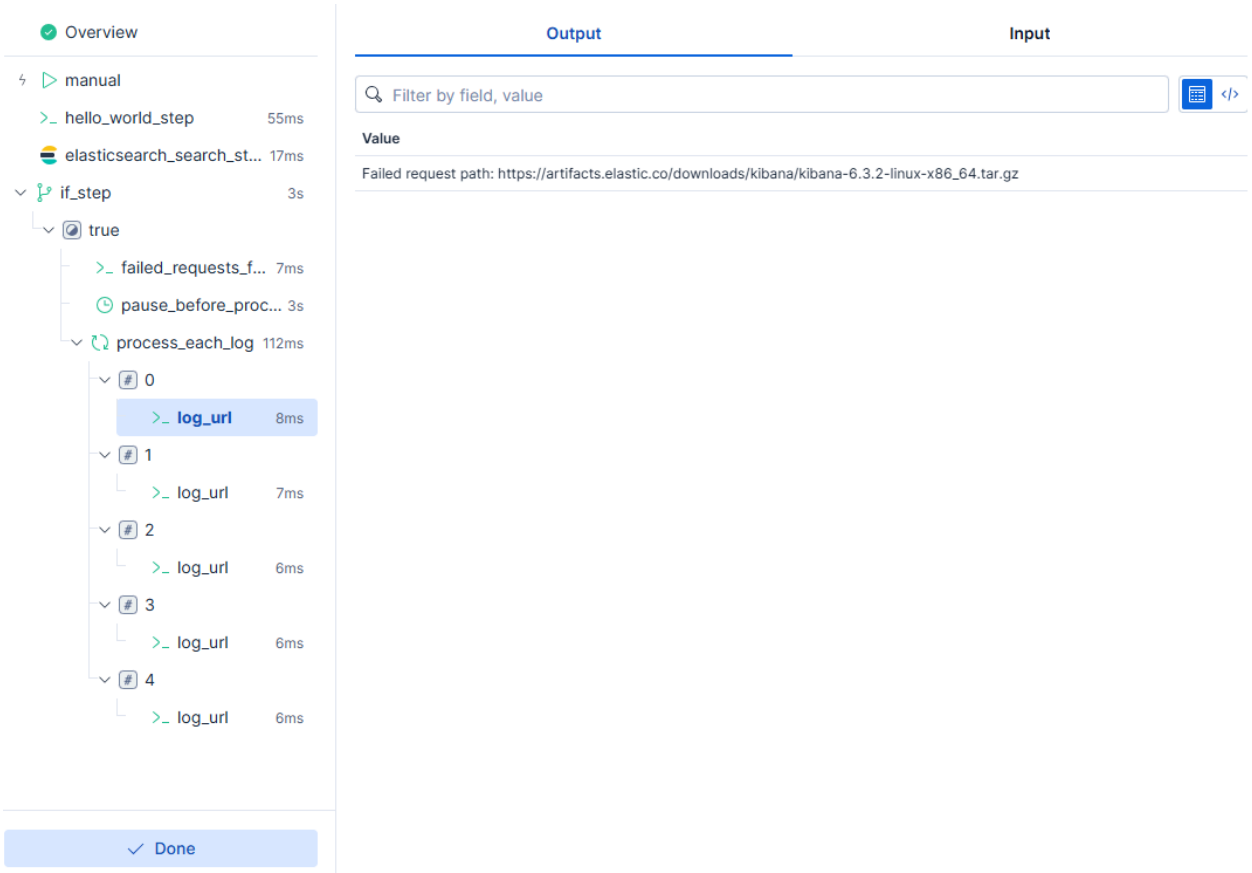
- Expand the **if_step**
- Expand the true branch

- Click **failed_requests_found** and confirm it displays the number of failed requests returned by the search
- Click **pause_before_processing** and confirm the timeline shows a 3-second delay
- Expand **process_each_log** to see each loop iteration
- Click any **log_url** entry and confirm it displays a failed request path from the sample data




kubernetes-vm-30001-chlfyzwlu75k.env.play.instruqt.com/app/workflows/workflow-5d47cc16-c227-49c1-a749-d0de2783a0bb
elastic
Find apps, content, and more.
AI Assistant
Workflows
my-first-workflow
Back to Workflows
my-first-workflow
Unsaved changes
Workflow
Executions
Enabled
Save
size: 5
- name: if_step
type: if
condition: steps.elasticsearch_search_step.output.hits.total.value > 0
steps:
- name: failed_requests_found
type: console
with:
message: "Found {{ steps.elasticsearch_search_step.output.hits.total.value }} failed requests. Processing the first 5 results..."
- name: pause_before_processing
type: wait
with:
duration: "3s"
- name: process_each_log
type: foreach
foreach: "{{ steps.elasticsearch_search_step.output.hits.hits | json }}"
steps:
- name: log_url
type: console
with:
message: "Failed request path: {{ foreach.item._source.url }}"
else:
- name: no_failed_requests_found
type: console
with:
message: "No failed requests were found in the sample web logs."
1 warning
Actions menu Ctrl + K


kubernetes-vm-30001-chlfyzwlu75k.env.play.instruqt.com/app/workflows/workflow-5d47cc16-c227-49c1-a749-d0de2783a0bb?executionId=78a0db8c-4521-4b77-8494-97835164c607&stepExecutionId=__overview
elastic
Find apps, content, and more.
AI Assistant
Workflows
my-first-workflow
Back to Workflows
my-first-workflow
Saved just now
Workflow
Executions
Enabled
Save
Overview
manual
hello_world_step 55ms
elasticsearch_search_st... 17ms
if_step 3s
Done
Success
just now
3s
Execution started
Oct 7 @ 2026, 19:50:33.653
Execution ended
Oct 7 @ 2026, 19:50:37.037
Filter by field, value
Field
Value
spaceId default
execution.isTestRun true
execution.startedAt Oct 7, 2026 @ 19:50:33.653
execution.id 78a0db8c-4521-4b77-8494-97835164c607
execution.url http://localhost:30002/app/workflows/workflow-5d47cc16-c227-49c1-a749-d0de2783a0bb?executionId=78a0db8c-4521-4b77-8494-97835164c607&tab=executions
workflow.spaceId default
workflow.name my-first-workflow
workflow.id workflow-5d47cc16-c227-49c1-a749-d0de2783a0bb
workflow.enabled false
kibanaUrl http://localhost:30002
now Oct 7, 2026 @ 19:50:37.037
consts -
trace.traceId 35e676f17d1898c48ab35c6c0ba3a4b8
trace.entryTransactionId 25989df47e0d9c07



In this lesson:

What I Learned — Elastic Stack Workflows

In this lesson, I learned how to build a **dynamic Elastic workflow** that can search Elasticsearch data, check the results, pause execution, and process each result individually.

The main idea is:

Trigger → Action → Search → Check result → Wait → Process results

---

1. I learned what an Elastic Workflow is

A workflow is a series of steps that Elastic executes in a specific order.

For example:

```
Workflow starts
      ↓
Search Elasticsearch
      ↓
Check if results exist
      ↓
If results exist
      ↓
Process the results
      ↓
Finish
```

Workflows can be used to automate tasks that a SOC analyst would otherwise perform manually.

---

2. I learned about Workflow Triggers

A **trigger** determines **when the workflow starts**.

In my lab, I used:

```
triggers:
  - type: manual
```

This means I manually click **Run** to start the workflow.

In a real SOC environment, workflows can be used with security events and alerts to automate responses.

---

3. I learned about Inputs

Inputs are information provided to a workflow.

Example:

```
inputs:
  - name: message
```

```
    type: string
    default:"This is the default message"
```

Here:

- `message` = name of the input
- `string` = the input contains text
- `default` = value used when no other value is provided

The input can then be used with:

```
{{ inputs.message }}
```

---

4. I learned about Action Steps

Action steps perform actual tasks.

For example:

```
type: elasticsearch.search
```

performs a search in Elasticsearch.

Another example:

```
type: console
```

displays information in the workflow execution panel.

So I learned:

> **Action steps are the parts of the workflow that actually do something.**

---

5. I learned how to search Elasticsearch from a workflow

I created an Elasticsearch search step:

```
- name: elasticsearch_search_step
  type: elasticsearch.search
  with:
    index:"kibana_sample_data_logs"
    query:
      match:
        response:"404"
    size: 5
```

This searches the:

```
kibana_sample_data_logs
```

index.

It looks for:

```
response = 404
```

A **404** usually means that a requested web page/resource was not found.

I also used:

```
size: 5
```

which means:

> Return only the first 5 matching documents.

This makes the workflow easier to understand while learning.

---

6. I learned about Workflow Outputs

When an action runs, it can produce an **output**.

For example, the Elasticsearch search produces information such as:

```
hits
 └── total
      └── value
```

This:

```
steps.elasticsearch_search_step.output.hits.total.value
```

means:

> Go to the output of the Elasticsearch search step and get the total number of matching documents.

For example:

```
hits.total.value = 25
```

means:

> Elasticsearch found 25 matching documents.

---

7. I learned Template Syntax

Elastic workflows use expressions such as:

```
{{ ... }}
```

For example:

```
{{ steps.elasticsearch_search_step.output.hits.total.value }}
```

This means:

> Take the value produced by the Elasticsearch search step.

I can use this value in another step.

Example:

```
message:"Found {{ steps.elasticsearch_search_step.output.hits.total.value }} failed reque
sts"
```

If Elasticsearch found 25 results, the output becomes:

```
Found 25 failed requests
```

So I learned that **one step can use the output of another step**.

---

8. I learned Flow Control

Flow control determines **what the workflow should do next** based on a situation.

The main flow-control steps I learned are:

```
IF
WAIT
FOREACH
```

These make the workflow dynamic instead of simply executing every step one after another.

---

9. I learned the `if` step

The `if` step checks a condition.

I used:

```
- name: if_step
  type: if
  condition: steps.elasticsearch_search_step.output.hits.total.value > 0
```

This asks:

> **Did Elasticsearch find any results?**

The condition is:

```
total > 0
```

There are two possible outcomes:

```
             Search
                ↓
        Are there results?
           /          \
        YES            NO
         ↓              ↓
      TRUE           ELSE
      branch         branch
```

---

10. I learned the True Branch

If the condition is true:

```
hits.total.value > 0
```

the workflow enters the **true branch**.

I used:

```
message:"Found ... failed requests. Processing the first 5 results..."
```

So if results exist, the workflow tells me that failed requests were found.

---

11. I learned the Else / False Branch

If there are no results:

```
hits.total.value = 0
```

the workflow goes to the `else` branch.

I used:

```
else:
  - name: no_failed_requests_found
    type: console
    with:
      message:"No failed requests were found in the sample web logs."
```

So the workflow can handle **both situations**.

Results found:

```
Found failed requests
```

No results:

```
No failed requests were found
```

This is much better than having a workflow that assumes results will always exist.

---

12. I learned the `wait` step

I added:

```
- name: pause_before_processing
  type: wait
  with:
    duration:"3s"
```

This pauses the workflow for **3 seconds**.

So:

```
Find results
     ↓
Wait 3 seconds
     ↓
Process results
```

The `wait` step can be useful when I need the workflow to pause before continuing.

---

13. I learned the `foreach` step

This is one of the most important concepts.

`foreach` means:

> **Do something for every item in a list.**

For example, imagine Elasticsearch returns:

```
Result 1
Result 2
Result 3
Result 4
Result 5
```

Instead of processing everything together:

```
foreach
  ↓
Result 1 → process
Result 2 → process
Result 3 → process
Result 4 → process
Result 5 → process
```

So `foreach` is basically a **loop**.

---

14. I learned what `foreach.item` means

Inside the loop, I used:

```
{{ foreach.item._source.url }}
```

`foreach.item` means:

> **The current result being processed.**

For example:

```
Result 1
   ↓
foreach.item
```

Then:

```
foreach.item._source.url
```

gets the URL from that particular result.

So the workflow can output:

```
Failed request path: /login
Failed request path: /admin
```

```
Failed request path: /test
Failed request path: /unknown
```

Each result is processed separately.

---

15. I learned how the complete workflow works

My workflow now works like this:

```
            MANUAL TRIGGER
                  ↓
          Hello World Step
                  ↓
       Elasticsearch Search
                  ↓
     Search for response: 404
                  ↓
         Are results > 0?
            /        \
           YES        NO
            ↓          ↓
   Failed requests    No failed
        found         requests
            ↓
       Wait 3 seconds
            ↓
    Process each result
            ↓
      Get current URL
            ↓
   Display failed request
```

---

16. What I learned about SOC automation

This is especially useful for my **SOC Analyst L1** learning.

Instead of manually doing:

```
Receive alert
     ↓
Search Elasticsearch
     ↓
Check results
     ↓
Look at each result
     ↓
Investigate
```

a workflow can automate some of these tasks.

For example:

```
🚨 Security Alert
       ↓
Search related logs
       ↓
Are there matching events?
      / \
    YES  NO
     ↓    ↓
Process  Close/
events   finish
     ↓
Extract useful information
     ↓
Notify SOC analyst
```